

Real-Time Telemetry & Tactical Comm-Link Analyzer

Basic Skeleton Generation Prompt for Antigravity IDE

You are an expert system architect and full-stack developer. Generate the foundational skeleton for a high-performance, real-time data fusion system. This system ingests high-frequency vehicle telemetry (100Hz+) via WebSockets, processes asynchronous audio streams via a decoupled NLP worker, and fuses the data based on source-level UTC timestamps.

Tech Stack Requirements

- **Backend / API Gateway:** FastAPI (Python), Uvicorn, WebSockets.
- **State / In-Memory Store:** Redis (with TimeSeries module if available, otherwise simulate a rolling window buffer).
- **NLP/DL Worker (Mocked for now):** Celery or FastAPI Background Tasks.
- **Frontend Dashboard:** React (TypeScript), Zustand (State Management), Recharts (or similar for time-series visualization).

Scaffold the following directory structure and core files:

1. docker-compose.yml

Set up the fundamental services:

- `api` : The FastAPI backend.
- `worker` : A separate Python service for async NLP tasks.
- `redis` : The Redis instance (expose port 6379).
- `frontend` : The React application.

2. Backend (FastAPI)

Create the `backend/` directory with the following structure:

- `backend/main.py` : Initialize the FastAPI app. Implement two primary endpoints:
 - A WebSocket route (`/ws/telemetry`) to ingest high-frequency JSON data.
 - A REST endpoint (`/api/audio-event`) to receive an audio event payload (`start_ts` , `end_ts` , `mock_transcript`).
- `backend/redis_client.py` : Basic connection setup to the Redis instance.
- `backend/fusion_engine.py` : A stub function `fuse_data(start_ts, end_ts, transcript)` that represents querying Redis for telemetry within that time window and emitting a combined JSON object.

3. NLP Worker (Python)

Create the `worker/` directory with the following structure:

- `worker/tasks.py` : Create a mock background task (using Celery or standard Python `asyncio.sleep` to simulate a 3-second delay). It should receive an audio event, wait, and then call back to the main backend API or write directly to a shared Redis pub/sub channel indicating the "transcript" is ready.



4. Frontend (React + TypeScript)

Create the `frontend/` directory (standard Vite/Create React App structure). Focus on:

- `frontend/src/store/useStore.ts` : Initialize a Zustand store to handle high-frequency incoming data without re-rendering the entire app. It needs state for `liveTelemetry` and `fusedInsights` .
- `frontend/src/App.tsx` :
 - Establish a WebSocket connection to the backend on mount.
 - Render a mock button: "Simulate Radio Transmission".
 - Render two sections: A "Live Telemetry" placeholder chart and a "Tactical Insights" feed that displays the fused data when the backend pushes it.

Coding Standards

- Use strict type hinting in Python (`pydantic` models for incoming payloads).
- Use TypeScript interfaces for all frontend data structures.
- Ensure the WebSocket implementation in FastAPI uses `async` handlers.
- Provide clear inline comments explaining *where* the actual ML models (Whisper, Autoformer) will be injected later.